CIS 1904: Haskell

Building GPT-2 from Scratch in Haskell
(Experimental Lecture)

Agenda

- Motivation and introduction for today
- Gradient descent, and how computers try to do it (automatic differentiation)
- Continuation Passing Style (style of functional programming)
- Intro (but not full intro) to the transformer model
- Brief intro to programming languages and machine learning research

Please pull the latest of the class repository!

ALSO YOUR HOMEWORK IS NOT RELATED TO THIS LECTURE

Motivation - GPT2

microgpt.py (from Andrej Karpathy)

We will only work through some parts of this. Transformers are complicated and they take time to learn (and you can definitely learn them).

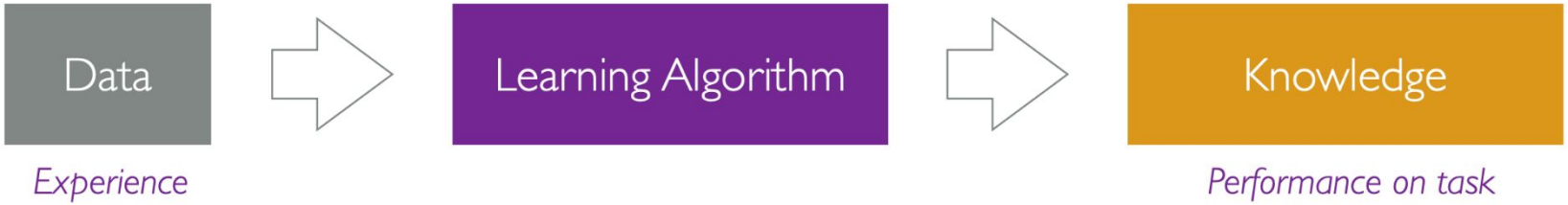
```python
import torch.nn as nn
import torch.nn.functional as F


class Net(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1) # flatten all dimensions except batch
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x


net = Net()
```

https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

What is machine learning?

Design and analysis of algorithms that

- Improve **performance**
- at some **task**
- given **experience**



This slide is from Eric and Surbhi's CIS 5200 class

Weights

Suppose you want to predict house prices from square footage and you have historical pricing data.

Price = weight x house size + bias

Weight - controls how strongly input affects outputs (imagine knobs in your car)

Bias - constant that gets added regardless of input. Think of it as a baseline - houses are at least $50,000 baseline for ex.

Super high level overview on how learning works

1. We start with a model - it will be a function that accepts parameters, and inside our function we initialize weights that are multiplied against the parameters.
2. Given input x (training data), model predicts its outcome.
3. We compare outcome to the ground truth using a loss function.
4. We try to minimize the error in step 2 by changing the weights.
5. Repeat

Gradient Descent (analogy first)

How do we minimize the loss? Gradient descent.

We compute the partial derivatives of the loss with respect to each weight and with respect to b. This means - **if I change a given weight by a small amount, how much does the loss change, per unit of that change?**

In computers, we store these derivatives in a vector. A gradient is a vector of all of these partial derivatives stacked together.

Backpropagation (how we calculate the partial derivatives)

Remember those partial derivatives we had to compute? We need to compute these. Backpropagation is probably the most standard way of doing this. Thanks to the chain rule we can go backwards from the loss and reuse our computation.

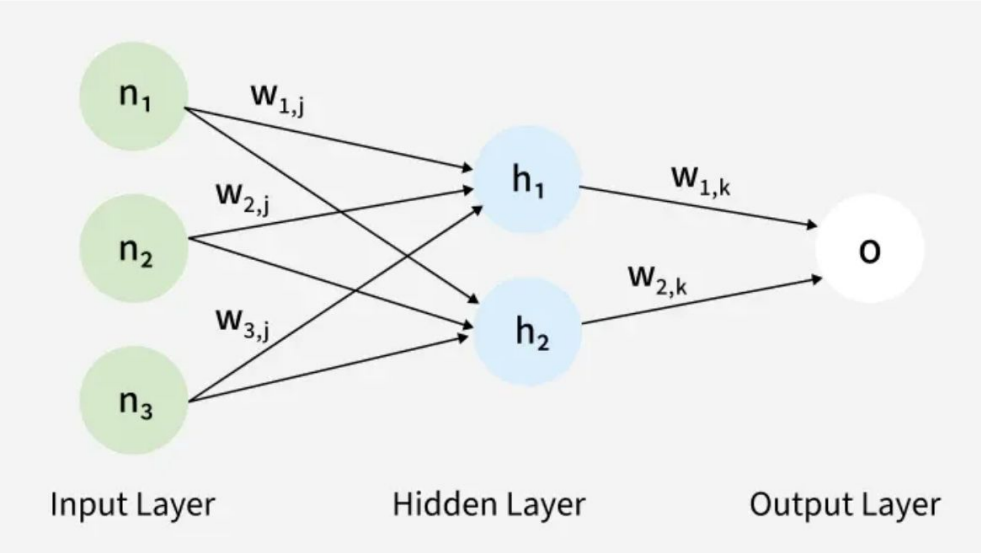


Image source: https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/

Gradient descent update step (this is where we minimize for real)

New weight = old weight - (learning rate x gradient)

Learning rate - how big of a step the model takes each time it updates its weights

Ex . learning rate = 0.001, 0.01 ..

Automatic Differentiation

- PyTorch Autograd
- What are potential issues that you see with implementing automatic differentiation in haskell?

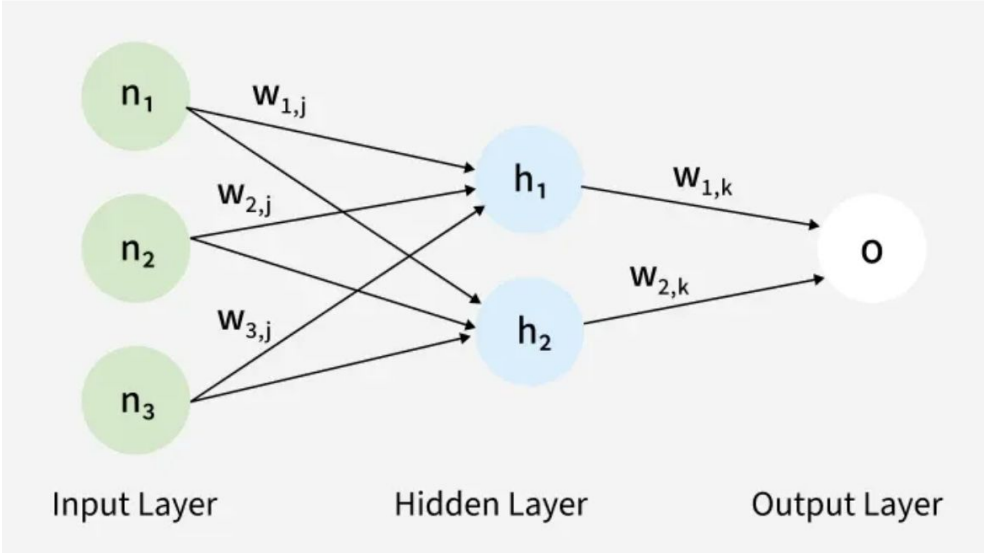


Image source: https://www.geeksforgeeks.org/machine-learning/backpropagation-in-neural-network/

Continuation Passing Style (CPS)

- We will not get into this implementation, but this is how ad (popular haskell automatic differentiation library implements reverse).
- Continuation - function representation of the execution state of the program at a certain point in time
- Instead of a function returning its result, you pass the result as an argument to the continuation function.

```
add : Int -> Int -> int
add x y = x + y
```

```
addCPS :: Int -> Int -> (Int -> r) -> r
addCPS x y k = k (x + y)
```

CPS

```
addCPS :: Int -> Int -> (Int -> r) -> r
addCPS x y k = k (x + y)
```

Compute (3 + 4) + 5

```
example :: (Int -> r) -> r
example k =
    addCPS 3 4 (\sum1 ->
        addCPS sum1 5 (\sum2 ->
            k sum2)

main = print (example (\result -> result - 2)
```

CPS

```
addCPS :: Int -> Int -> [(String, Int, Int)] -> ([(String, Int, Int)] -> Int
-> r) -> r
addCPS x y tape k =
    let result = x + y
    tape’ = tape ++ [(“add”, x, y)]
    in k tape’ result

Compute (3 + 4) + 5

example :: ([(String, Int, Int)] -> Int -> r) -> r
example k =
    addCPS 3 4 [] (\tape1 sum1 ->
        addCPS sum1 5 tape1 (\tape2 sum2 ->
            k tape2 sum2)

main = print (example (\tape result ->
    do print result
    print tape)
```

Quick CPS Exercise

```
subtractCPS :: Int -> Int -> (Int -> r) -> r
```

Combine addCPS and subtractCPS to compute (3 + 4) - 2

```
exampleAddSubtract :: (Int -> r) -> r
```

Building GPT-2 from scratch

- GPT-2: transformer style large language model by OpenAI
- Python Example
- You can refer to Karpathy's implementation if that helps. I've also included my own full implementation. In fact, open up both the python file and MicroExercise.hs!
- What are some parts in microgpt.py that you know you'll have to write differently in Haskell?

Quick Note

1. Why is there a Value ADT instead of just dealing with existing number primitive types? So we can create a graph structure where when we traverse it backwards we can reuse gradients that we computed.
2. Why did you define this strange StateIO Monad? I was using IO monad due to using a random number generator library, and State monad for keep track of the gradients. A way to combine these two is actually using a monad transformer, which is what you'll learn on the last day.

Auto differentiation

Given the provided ADT Value, used to store gradients, fill in the code to build the gradient graph, used in backpropagation.

```
topologicalSort :: Set.Set -> Int -> Value -> ([Value],
Set.Set Int)
```

ReLU

f(x) = max (0, x)

Super common function found all over neural networks. It helps introduce non-linearity, which helps neural networks learn complex patterns. (What happens if we don’t have non-linearity?)

```
reluV :: Value -> Trace Value
```

Linear

- Linear transformation takes a vector in one space and maps it to another space
- Probably the most common function that shows up in all kinds of neural nets.
- Linear takes an input vector and weight matrix. For each row of the weight matrix, it computes the dot product with the input (multiply its elements element wise with the input vector, then sum the results into a single value)

```
linear :: [Value] -> [[Value]] -> Trace [Value]
```

RMS Norm

- Root mean squared normalization - normalizes a vector by by using the root mean square of its elements
- This keeps the scale of activitations (any computation of input multiplied by weight) consistent throughout training.

```
rmsNorm :: [Value] -> Trace [Value]
```

$$y_i = \frac{x_i}{\mathrm{RMS}(x)} * \gamma_i, \quad \text{where} \quad \mathrm{RMS}(x) = \sqrt{\epsilon + \frac{1}{n}\sum_{i=1}^{n} x_i^2}$$

Softmax

- How we convert raw logits to probabilities

```
softmax :: [Value] -> Trace [Value]
```

$$\mathrm{softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_j e^{x_j - \max(x)}}$$

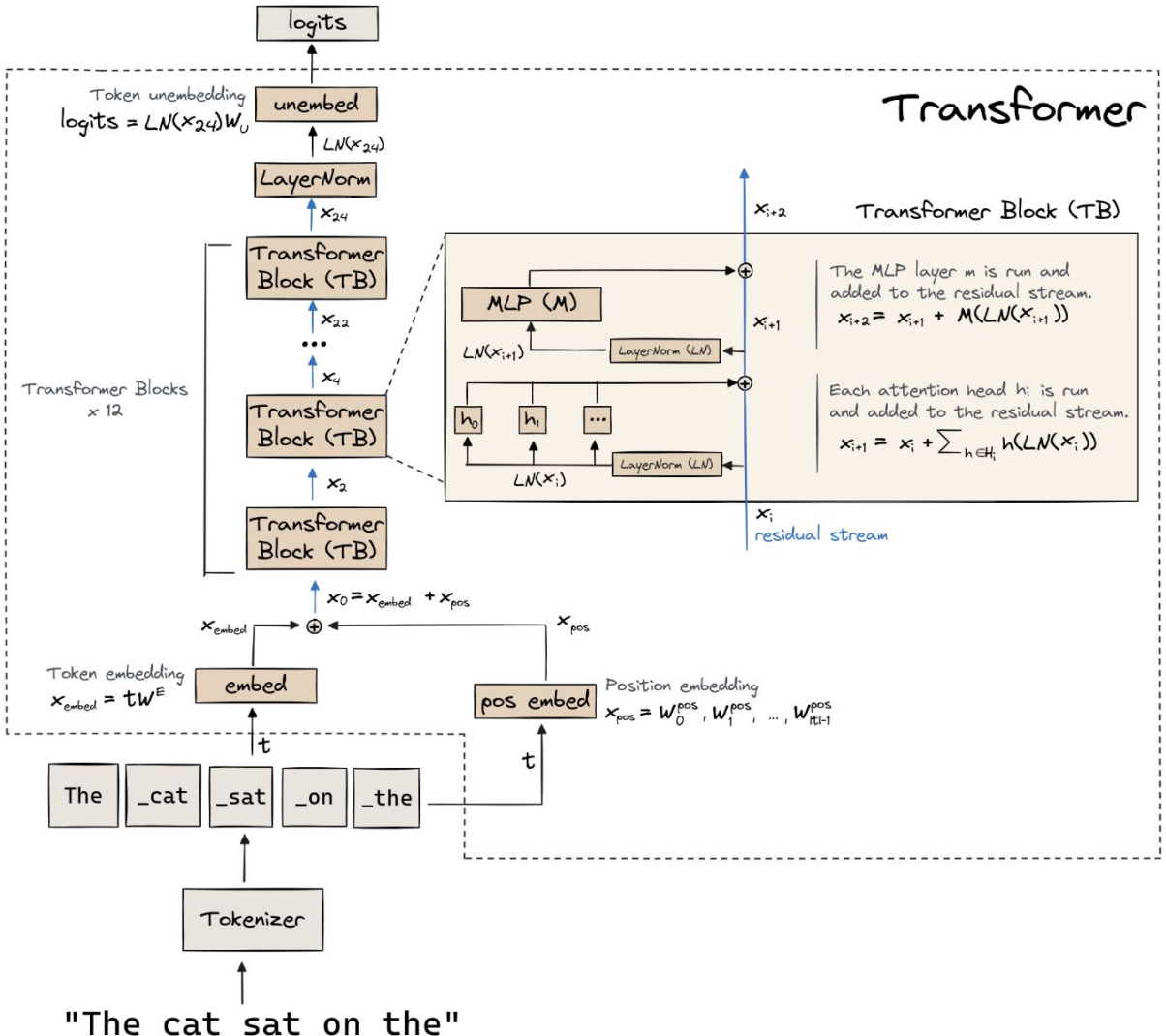
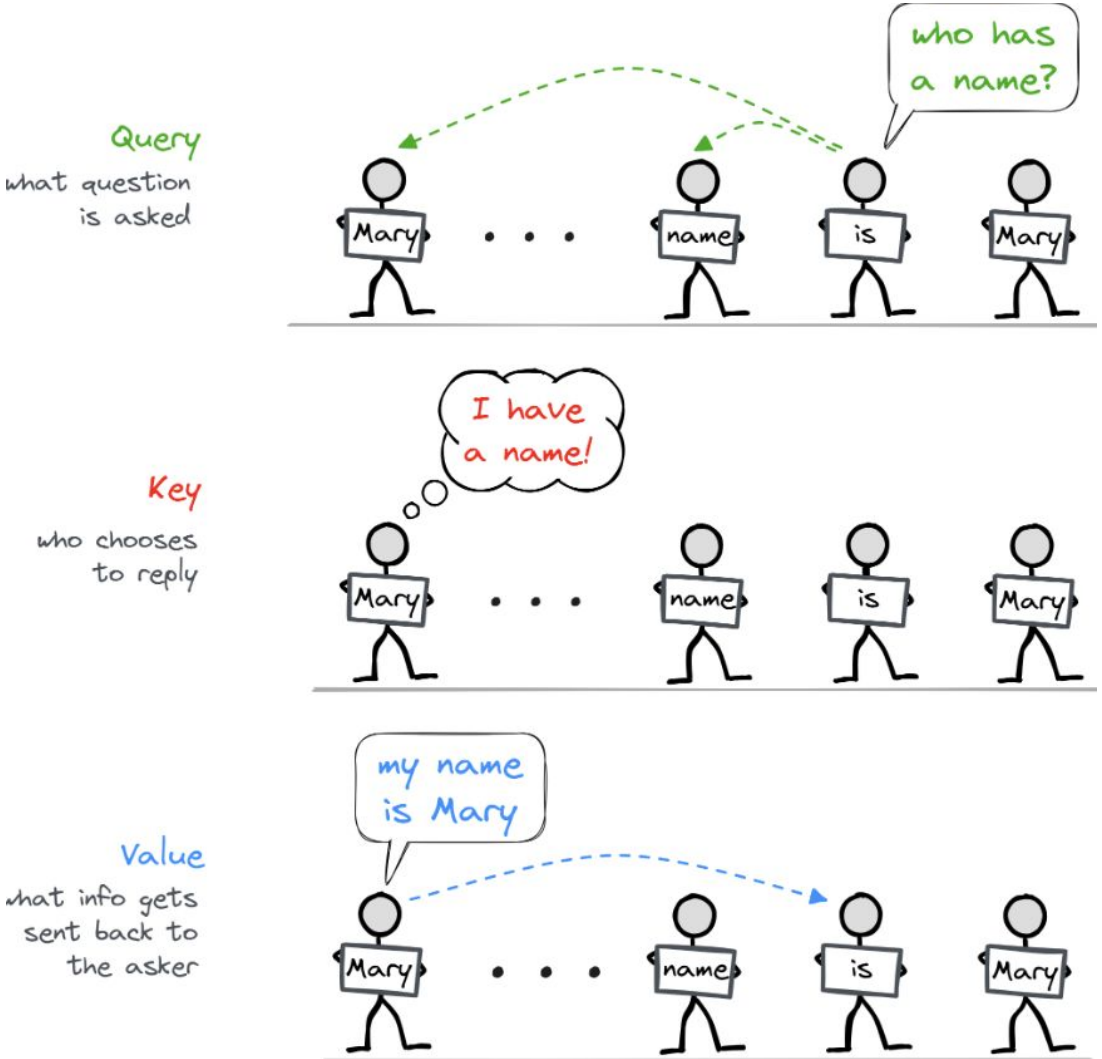


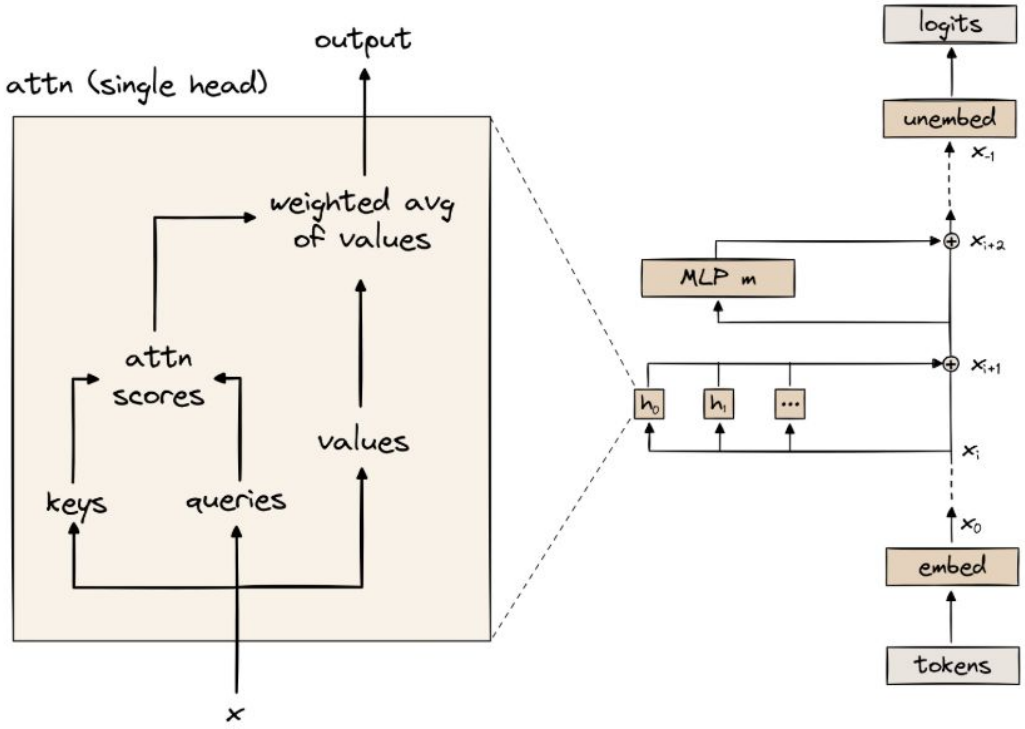
Image from Transformers from scratch, ARENA https://learn.arena.education/chapter1_transformer_interp/01_transformers/intro/

Attention



https://learn.arena.education/chapter1_transformer_interp/01_transformers/intro/

Attention Head



https://learn.arena.education/chapter1_transformer_interp/01_transformers/intro/

That was a lot of information (takes time to learn)

- 3Blue1Brown has a popular Neural networks playlist https://www.youtube.com/watch?v=aircAruvnKk&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi
- Try using existing transformer libraries https://pytorch.org/hub/huggingface_pytorch-transformers/
- Read popular transformer papers such as Attention is All You Need (Vaswani et al. 2017) https://arxiv.org/abs/1706.03762 and try cloning their code
- Wikipedia page on the history of neural networks https://en.wikipedia.org/wiki/History_of_artificial_neural_networks - helps understand how we have evolved architectures

Programming Languages and Machine Learning Research

- Exciting new intersection given that LLMs are non-deterministic black boxes, and programming languages provide us formal methods, verification, and testing.
- My main dissertation question: What happens when everyone is using LLM coding assistants? What new problems arise?
- In general, try reading and implementing papers. Reach out to professors at Penn who you find interesting. Pursuing CS research in this current day is exciting!
- If you’re interested in pursuing this kind of research/future, feel free to reach out to me - cdwang@seas.upenn.edu

What next?

- Make a faster version of this. See nanoGPT leaderboard for inspo. Currently we don't have a leaderboard for fastest Haskell GPT-2 implementation - that could be you!
- Jax - Array computation library (comparable to Numpy) with a functional programming paradigm
- What are some fp ideas that you don't see currently adapted in today's machine learning research, but could be interesting?
- If you're interested in continuation passing style - this is a beautiful field of PL research with applications to compilers, and an interesting correspondence to sequent calculus .